

**INSTITUTO
FEDERAL**
Santa Catarina

Introdução ao TypeScript

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO
FRONTEND II

Análise e Desenvolvimento
de Sistemas

cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



TypeScript

- superconjunto do js
- adiciona tipagem estática e orientação a objetos ao js
- detecta erros precocemente
- melhora a legibilidade e a manutenção do código
- contribui para a escalabilidade do projeto
- aprimora a contribuição de IDEs



Alternativas ao TypeScript



CoffeeScript

- lançada em 2009
- alternativa mais concisa e expressiva ao JavaScript
- tornar a sintaxe do JS mais parecida com a do Ruby ou do Python



Dart

- lançada em 2011
- alternativa ao JS para aplicativos web, móveis e desktop
- sintaxe semelhante ao Java e ao C#



elm

- lançada em 2012
- projetado para a construção de aplicações web
- modelo de programação funcional
- sintaxe inspirada no Haskell



Preparação

pré-requisito node.js com npm

instalação global

1. instalar typescript via npm
 - `npm install -g typescript`
2. testar se está tudo certo
 - `tsc --version`

instalação local

1. instalar typescript via npm
 - `npm install typescript`
2. adicionar o compilador à lista de scripts em `package.json`
 - ```
"scripts": {
 "tsc": "tsc"
}
```
3. testar se está tudo certo
  - `npm run tsc -- --version`



# Preparação

## configuração

### 1. criar arquivo tsconfig.json

- manualmente
- automático
  - `npx tsc --init`

### 2. instalar/atualizar os módulos do projeto

- `npm install`

## exemplo

### tsconfig.json

```
{
 "compilerOptions": {
 "target": "ES2016",
 "outDir": "./dist/js",
 "noEmitOnError": true,
 "module": "commonjs",
 "moduleResolution": "node",
 "strict": true,
 },
 "include": [
 "./src/**/*.ts"
]
}
```



# Preparação

## compilação

### 1. por arquivo

- `tsc arquivo.ts` (global)
- `npm run tsc arquivo.ts` (local)

### 2. automática

- `tsc -w` (global)
- `npm run tsc -- -w` (local)



# Tipos

## definição

- um conjunto de valores e o que pode ser feito com eles

## exemplos

- booleano
  - **valores** true, false
  - **operações** ||, &&, !, etc
- número
  - **valores** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
  - **operações** +, -, \*, /, %, .toFixed, .toPrecision, .toString, etc
- string
  - **valores** *todas as strings*
  - **operações** +, ||, &&, .concat, .toUpperCase, etc





# Tipos

## definição

- um conjunto de valores e o que pode ser feito com eles
- podem ser definidos explicitamente
  - valor: tipo
- podem ser inferidos

## exemplos

```
// tipos definidos explicitamente
let a: number = 1
let b: string = 'hello'
let c: boolean[] = [true, false]
```

```
// tipos inferidos
let a = 1
let b = 'hello'
let c = [true, false]
```

# Tipos

## primitivos

### ■ any

```
let a: any = 666
let b: any = ['perigo']
let c = a + b // atribuicao implicita
```

### ■ unknown

```
let a: unknown = 30
let b = a === 123
let c = a + 10 // Error TS2571: Object is of type
'unknown'.
```

```
if (typeof a === 'number') {
 let d = a + 10 // number
}
```

### ■ boolean

```
let temSaldo: boolean = true;
let verdadeiro: true = true;
```

### ■ number

```
let numero = 10;
let idade: number = 34;
let media: 6.5 = 6.5;
```

### ■ string

```
let nome = 'João';
let sobrenome: string = 'Silva';
```

# Objetos

## declarando

### ■ anotando como object

```
let obj: object = { prop: 12 };
obj.prop; // Error TS2339: Property 'prop' does not
exist on type 'object'.
```

### ■ como object literal

```
let obj = { prop: 12 };
console.log(obj.prop); // 12
obj.outraProp = 123; // Error TS2339: Property
'outraProp' does not exist on type 'object'.
```

### ■ definindo o formato do objeto

```
let obj: {
 prop: number // obrigatória
 nome?: string // opcional
 [chave: number]: boolean // qualquer quantidade;
 [chave: T]: (index signature)
}
obj = { prop: 12, nome: 'João', 1: true, 2: false }
```

### ■ como objeto vazio

```
let obj = {};
obj = { prop: 12 };
obj = { prop: 12, nome: 'João' };
```



# Tipos Personalizados (Type Aliases)

## o que são

- "apelidos" para tipos
- ajudam no entendimento
- não são inferidos; devem ser definidos explicitamente
- podem ser substituídos pelo tipo original
- escopo de bloco

## exemplos

```
type Matricula = number
let novaMatricula: Matricula = 202421232;

type Pessoa = {
 matricula: Matricula;
 nome: string;
 funcao: string;
}

let novoAluno: Pessoa = {
 matricula: novaMatricula,
 nome: 'João',
 funcao: 'Aluno'
}
```



# União e Interseção

## união

- operador |
- permite que uma variável aceite um valor de mais de um tipo

```
let id: string | number;
```

```
id = "ABC123"; // Válido
```

```
id = 456; // Também válido
```

## interseção

- operador &
- combina múltiplos tipos em um único tipo

```
type Pessoa = {
 nome: string;
 idade: number
};
```

```
type Funcionario = {
 id: number;
 departamento: string
};
```

```
type PessoaFuncionario = Pessoa & Funcionario;
```

```
const funcionário: PessoaFuncionario = {
 nome: "Alice",
 idade: 30,
 id: 123,
 departamento: "Engenharia"
};
```



# Arrays

## declarando

### ■ sem anotação (tipo inferido)

```
let numeros = [1, 2, 3];
var letras = ['a', 'b'];
```

### ■ anotando o tipo

```
let numeros: Array<number>;
var letras: Array<string>;

// -----
let numeros: number[];
var letras: string[];
```

### ■ vazio

```
let vazio = []; // any[]
vazio.push(1); // number[]
vazio.push('a'); // (number | string)[]
```

### ■ apenas leitura

```
let numeros: readonly number[] = [1, 2, 3];

numeros.push(4); // Error TS2339: Property 'push'
does not exist on type 'readonly number[]'.
```



# Enumeráveis (Enums)

## o que são

- conjunto de valores constantes
- estruturas de dados não ordenadas que mapeiam chaves para valores
- os valores podem ser do tipo number ou string

## exemplos

```
enum Idioma {
 INGLES, // 0
 ESPANHOL, // 1
 RUSSO // 2
}
```

```
enum Idioma {
 INGLES = 0,
 ESPANHOL = 1,
 RUSSO = 2
}
```

```
enum Idioma {
 INGLES = "inglês",
 ESPANHOL = "espanhol",
 RUSSO = "russo"
}
```



# Enumeráveis (Enums)

## o que são

- conjunto de valores constantes
- estruturas de dados não ordenadas que mapeiam chaves para valores
- os valores podem ser do tipo number ou string

## exemplos

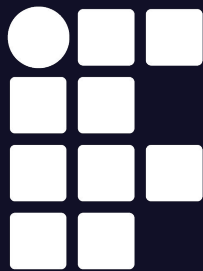
```
enum TipoFuncao {
 ALUNO = 'Aluno',
 PROFESSOR = 'Professor',
 FUNCIONARIO = 'Funcionário'
}

type Pessoa = {
 matricula: 202421232;
 nome: string;
 funcao: TipoFuncao;
}

let novoAluno: Pessoa = {
 matricula: 123,
 nome: 'João',
 funcao: TipoFuncao.ALUNO
}
```







**INSTITUTO  
FEDERAL**  
Santa Catarina

# Introdução ao TypeScript

## Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

[adriano.lima@ifsc.edu.br](mailto:adriano.lima@ifsc.edu.br)



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PROGRAMAÇÃO  
FRONTEND II

Análise e Desenvolvimento  
de Sistemas